Seam-to-Ledger Map

Module 5: Seam Ledger (Contested Attestation Ledger) — design artifact 1 of 4 Prepared 2026-07-14, against kernel **v0.12** (source digest `88ea6583…`) and the Veriticide General Ledger at `60aa8a6` (Documentation & Standing Protocol v0.1; Provenance Grading & Corpus-Absorption Protocol 2026-07-06; case-file grammar `cases/*/00…05`). Targets kernel **v0.13+**; introduces no change to v0.12.

---

0. What this map is, and the property class it serves

The v0.12 kernel is robust against **malformed** inputs: an under-provisioned sensor network is unconstructible (`Ensemble.quorum`), a certificate that grants an exposure-exiting action is unconstructible (`TEnvelope.route_preserves`), an emergency day during Structural Review is unconstructible (G2). It is **not** robust against **well-typed lies from legitimate authorities.** Its trusted inputs — the episode-machine signals `issue / confirm / novel / exceedance / layer0`, the danger threshold θ, certificate validity, the Module-4 criticality classification, and the Layer 0 dispositions and outputs — are **naked values an authority can cheaply assert.** Each is a `Bool`, an `Option`, or a `→ Bool` field whose *truth* the kernel never examines; it reasons only about what follows *given* the value.

Compressed thesis: AHC verifies consequences; the ledger contests premises. Module 5 imports the Veriticide ledger's *machinery* — not its corpus — so that every load-bearing external input becomes a **dated, attributed, contestable object** carrying evidence, counter-evidence, a falsifier, an authorization chain, and the affected population. The `SeamClaim α` wrapper replaces a naked `α` at the trust boundary.

The honest limitation, stated here and repeated in every Module 5 artifact: the ledger **does not catch lies.** It converts lying from a single administrative utterance into a *visible multi-party act with preserved contradictions.* No Module 5 theorem will assert that a wrapped payload is true. The deliberate omission from `SeamClaim` is a `truth : payload_is_true` field. The kernel proves **procedural** status only: provenance identified, evidence preserved, counter-evidence not erasable, issuer not sole validator, contestation visible, responses evidence-linked, no silent rewrites.

The map relocates trust boundaries; it does not eliminate them. Column (f) is therefore mandatory and non-decorative: every wrapper introduces at least one *new* oracle — an actor-identity relation, a custody attestation, an extraction judgment — whose truth the kernel again cannot check. Each such oracle is disclosed here as a first-class contestation target and, per the Provenance Protocol's **cost-to-fake rule** (Part II §5: *"before adding any new discriminator, state what it costs a bad-faith actor to fake; if the answer is 'nothing,' it is instrumentation, not evidence"*), carries a cost-to-fake analysis in column (d).

---

1. The wrapper (referenced by the map; full skeleton is artifact 2)

```
structure SeamClaim (α : Type) where
  payload           : α                      -- the naked v0.12 value, unchanged
  incidentId        : IncidentId
  issuer            : Actor
  authorizationChain : List Actor            -- Track B
  evidenceRefs      : List EvidenceRef       -- Track A (the spine)
  sourceBundleDigest : Digest                -- Track F / 02-source-bundle
  custodyStatus     : CustodyStatus          -- Track F ladder (§6)
```

```
  warningsAvailable  : List EvidenceRef        -- Track C
  counterEvidence    : List EvidenceRef        -- COUNTER-EVIDENCE STATUS field
  falsificationPath  : FalsificationCondition  -- 04-falsification-memo
  affectedCounterRecord : Option CounterRecord -- affected-population counter-register
  conflicts          : List Conflict
  status             : ClaimStatus
  -- deliberately absent: truth : payload_is_true
```

`payload` is the v0.12 input verbatim, so a `SeamClaim`-bearing interface can present the same downstream `EpInput` / `Envelope` / `Event` the current kernel consumes — this is what makes deprecation of the naked interfaces (boundary §6 of the handoff) a migration rather than a rewrite. **What Module 5 adds is a gate on *construction* of the wrapper, and theorems about what the gate guarantees — never a truth check on `payload`.**

Custody ladder (imported from `cases/*/05-custody-manifest.md`, unchanged in kind): `inLedger → captureRequired → locatorVerified → hashedPendingBackup → verified`. Only `verified` carries an off-platform second custodian; it is the sole tribunal-grade state.

---

2. The map

One row per trusted kernel input. Columns: **(a)** input as typed in v0.12 · **(b)** Standing-Protocol / case-file fields · **(c)** adversarial failure mode if the input stays naked · **(d)** cost-to-fake once the `SeamClaim` wrapper exists · **(e)** candidate Lean invariant (Tier A / Tier B per handoff §3) · **(f)** new oracle the wrapper itself introduces *(mandatory — a relocated trust boundary, disclosed as a contestation target).*

Row 1 — `issue : Bool` (emergency-authority initiation)

- **(a)** `EpInput.issue : Bool` — "request to issue a full PIO this hour" (`TieredProtocol.lean:770`); consumed by `estep .idle → .pending 0`.
- **(b)** Track A (Instrument & Outputs: the initiating output, verbatim, dated); Track B (Authorization: who issued it); `00-charge-theory` (the initiating act).
- **(c) The naked-authority-bit failure.** A single actor sets one Boolean and an emergency episode begins. There is no attribution, no date beyond the implicit hour, no evidence, and no record that anyone *other than the initiator* was involved. The emergency channel initiates from an unaccompanied bit — the paradigm the whole module exists to close.
- **(d)** With the wrapper, initiating an emergency requires forging a *populated* `SeamClaim Bool`: a non-empty `authorizationChain`, at least one `EvidenceRef`, a `sourceBundleDigest`, and a `custodyStatus`. Faking a bare bit costs nothing; faking a chain-of-approval plus dated evidence bundle costs a coordinated, attributable, later-auditable act by named roles. **The cost is not truth — it is *multi-party visibility*.** A false emergency can still be declared; it can no longer be declared *invisibly by one actor*.
- **(e)** *(Tier A)* `no_naked_authority_bit`: no `estep`/`epstep` emergency transition is reachable from a `SeamClaim` whose `authorizationChain = []` or `evidenceRefs = []` — such a claim is unconstructible at the issuing interface (a structure-field proof obligation, in the v0.12 house style).
- **(f) New oracle: the authorization-chain attestation.** Someone must attest that the listed `Actor`s in fact authorized the act. The kernel verifies a chain *exists and is non-empty*, never that its members truly approved. Cost-to-fake of the

oracle itself: a fabricated chain naming real roles is forgeable by one actor, but becomes a *documentary falsehood attributable to that actor* — which is the point. **Disclosed as contestation target M5-O1.**

Row 2 — `confirm : Bool` (Tier-1 correlational confirmation)

- **(a)** `EpInput.confirm : Bool` — "Tier-1 correlational confirmation arrived" (`TieredProtocol.lean:771`); `estep .pending → .confirmed`, handing the episode to the tier ladder.
- **(b)** Track A (the confirming measurement/finding, verbatim + source grade S/P/A); Track F (custody of the confirming artifact); `01-evidence-matrix` row.
- **(c)** A bare `true` promotes a pending PIO to `confirmed` and onto the enforcement ladder. The *confirming evidence* is invisible: no one can later check what correlational finding was claimed, who produced it, or whether it was preserved. This is the seam where a manufactured confirmation launders into authorized enforcement.
- **(d)** The wrapper forces `confirm` to carry `evidenceRefs` with a `custodyStatus` and `sourceBundleDigest`. Cost-to-fake rises from "flip a bit" to "produce a custody-bearing evidentiary reference that survives later inspection." Note the ledger's own grade ladder (S1/P1/P2/A1 in `01-evidence-matrix`) is the natural type for the evidence grade; a bare unsourced assertion is the weakest grade and is marked as such.
- **(e)** *(Tier A)* `confirmation_is_evidence_backed`: `.confirmed` is reachable only through a `SeamClaim` with `evidenceRefs ≠ []`. *(Tier B, optional)* a `custodyStatus ≥ locatorVerified` obligation for tier-ladder promotion, if the author wants custody to gate promotion (see decision D-3).
- **(f) New oracle: the extraction/grade attestation.** Someone judges that the referenced evidence *is* a Tier-1 correlational confirmation and assigns its grade. The kernel checks a reference of some grade *exists*, never that the grade is correct. **Contestation target M5-O2.** (This is the same class of seam Module 4 already names at claim-extraction — see Row 8.)

Row 3 — `novel : Bool` (materially-new-claim attestation, D-R4)

- **(a)** `EpInput.novel : Bool` — "ATG/Layer 0 attestation: materially new claim" (`TieredProtocol.lean:772`); with `issue`, the *only* signal that restarts a full PIO clock from `.hold`/`.spent` (`estep`, `epIsIssue`). Directly load-bearing: `novel_restart_from_spent`, `exceedance_cannot_restart`.
- **(b)** Track A (the assertedly-new claim, verbatim); Track C (Trajectory: prior instances — *novelty is exactly the negation of "this appeared before"*, so Track C's prior-instance record is its natural falsifier); `04-falsification-memo`.
- **(c)** `novel` is the reset key: a bare `true` reopens the full emergency clock that the hold/overdue budget machine exists to bound. An authority that can freely assert novelty can perpetually re-launch episodes, defeating the two-clock budget (E-series). Naked, "materially new" is unfalsifiable — no record of *what* is new or *against what baseline*.
- **(d)** The wrapper requires the novelty claim to name its baseline: a `falsificationPath` stating *what prior instance, if exhibited, refutes novelty*, plus `warningsAvailable`/Track-C references. Cost-to-fake rises from "assert new" to "assert new while naming the condition that would prove it old, on the record." A false novelty claim now ships with its own falsifier attached — the Provenance Protocol's discipline applied at the reset key.
- **(e)** *(Tier A)* `novelty_carries_falsifier`: a restart via `novel` is reachable only through a `SeamClaim` whose `falsificationPath` is non-trivial (not the vacuous `⊥`-condition). *(Composes with)* `exceedance_cannot_restart` (v0.12) — Module 5 adds that the *only other* restart path also cannot be a naked bit.

- **(f) New oracle: the baseline/prior-instance relation.** Judging whether a named prior instance is "the same claim" is an extraction judgment. The kernel checks a falsifier *is stated*, never that it is the *right* baseline or that the matching is correct. **Contestation target M5-O3.** Per boundary §3, this judgment must **not** be made by an LLM classifier — it is a human/Layer-0 extraction obligation.

Row 4 — `exceedance : Bool` + danger threshold θ (hazard signal)

- **(a)** `EpInput.exceedance : Bool` (`TieredProtocol.lean:773`) — "sensor signal above threshold"; in the bridge, **derived** from an f+1 sensor quorum against a free threshold θ (`SensorHour`, `ExceedanceBridge.lean`). Sustains the continuity-hold (`floor_persists`, X3 `hold_sustained_only_by_witnessed_danger`).
- **(b)** Track A (the sensor readings and threshold config, dated); Track C (warnings that the threshold was mis-set); Track F (custody of the sensor configuration).
- **(c) Partially closed already, and the map should say so honestly.** v0.6's bridge (X1–X4) already proves a captured minority of ≤ f sensors can neither fabricate nor solely sustain exceedance — this is the *template* for good design. **The residual naked input is θ itself:** the bridge header states θ "remains a free per-hour input that a deployment must bind to a fixed, versioned, publicly attested configuration." A legitimate authority who *sets θ* can manufacture or suppress exceedance by moving the threshold, entirely within the honest-sensor regime.
- **(d)** The wrapper applies to the **threshold configuration**, not the sensor bit (which the bridge already handles): θ becomes a `SeamClaim` carrying the versioned config digest, the authorization chain that set it, and `warningsAvailable` (dissent that θ is mis-calibrated). Cost-to-fake of a self-serving threshold rises from "pick a number" to "pick a number under a dated, attributed, publicly-attested configuration whose change is logged." This is the bridge header's own "binding θ" strengthening, supplied by Module 5 machinery.
- **(e)** *(Tier A)* `threshold_is_attested`: exceedance derivation consumes a `SeamClaim θ` with a bound `sourceBundleDigest` (versioned config), not a bare θ. Composes with X1–X4: honest-witness non-fabrication *plus* attested threshold.
- **(f) New oracle: the configuration-custody attestation.** Someone attests that the bound digest *is* the deployed configuration. The kernel checks a digest is bound, never that it matches the running system. **Contestation target M5-O4.** (This is a genuine relocation, not elimination: config-to-deployment binding is a custody question the ledger's Track F is built for but cannot make certain.)

Row 5 — `layer0 : Option Layer0Disposition` (typed review disposition)

- **(a)** `EpInput.layer0 : Option Layer0Disposition` (`TieredProtocol.lean:774`; `747`) — `continueHold | close | newEpisode`. From `.hold`/`.overdue`, a disposition takes priority over all other inputs (`estep`); `newEpisode` is the *only* path back to `.idle`; `close` parks in `.spent` (`close_cannot_launder`, `overdue_resolution_iff`).
- **(b)** Track B (who issued the disposition, and the chain); Track D (whether the disposition suppressed a live contestation); the two-step standing doctrine (§7): a disposition is a *step-two-adjacent* act.
- **(c)** The disposition is a naked authority verb. `close` on an `overdue` subgraph ends the flagged breach state; `newEpisode` re-arms a full clock. The *basis* of the disposition (v0.10's header: "recorded through D-R4's disclosure obligations, outside the machine") is not in the machine's type — so an interested actor can close out a contested hold with no attached reason, and no record of who they are relative to the contestation.

- **(d)** The wrapper carries `issuer`, `authorizationChain`, and — critically — the `conflicts` list and any `affectedCounterRecord`. Cost-to-fake of an interested close-out rises from "emit `.close`" to "emit `.close` while on the record as the issuer, with any live counter-record still attached and non-erasable." The disposition can still be wrong; it can no longer be *anonymous or contradiction-erasing*.
- **(e)** *(Tier A)* `counterrecord_persists`: once an `affectedCounterRecord` is attached to an incident, no `Layer0Disposition` transition erases it (a `close` parks the subgraph but the incident's counter-record survives into `.spent`). *(Tier A)* `remedy_label_not_remedy` (see Row 7). *(Tier B)* `interested_actor_cannot_self_close`: a disposition whose `issuer` stands in the conflict relation to the incident cannot be the sole closer (requires the actor relation — M5-O5).
- **(f) New oracle: the actor-identity/conflict relation.** Determining that an issuer is "interested" is itself an input the kernel cannot verify. Per boundary §3, **no LLM classifier decides this.** The kernel verifies the *structural* property (a second, non-conflicted validator exists / the counter-record survives), never the *substantive* judgment that a given actor is interested. **Contestation target M5-O5** — the single most consequential relocation in the module; Tier B rests on it.

Row 6 — `Layer0Output` (the Structural-Review closer)

- **(a)** `Layer0Output : remediationPlan | amendmentProcess | thresholdRecalibration` (`CrisisCap.lean:258`) — "the ONLY three" outputs that close a Structural Review (`review_exit_iff_output`, C5 `review_absorbing`, G3). `modeStep .structuralReview (.layer0 _) → .operational`.
- **(b)** Track B (who enacted the closing output); the two-step standing doctrine (§7, the step-one/step-two separation); Forum-Now toggle (§7-bis: closing review is a register-hardening *act*, logged).
- **(c) The remedy-label-is-not-remedy failure.** `remediationPlan` is a bare constructor: *enacting the label* closes review. There is no distinction between a plan that was *issued*, one whose *implementation began*, one whose *implementation was verified*, and one whose *effect was verified*. An authority closes the review by announcing a plan it never executes. (`thresholdRecalibration` compounds Row 4: closing review by moving θ.)
- **(d)** The wrapper types the remedy as a *lifecycle*, not a label: `planIssued | implementationBegun | implementationVerified | effectVerified | failed | contested`. Cost-to-fake of a hollow remedy rises from "name a plan" to "advance a dated lifecycle each stage of which is a separately attested, custody-bearing act." A review closed on `planIssued` is *visibly* closed on a mere plan; the record shows the distance from `effectVerified`.
- **(e)** *(Tier A)* `remedy_label_not_remedy`: a `remediationPlan` `SeamClaim` in state `planIssued` cannot exit review-satisfaction; the six-state lifecycle is distinct by construction (in the style of v0.8's `pio_roles_distinct` P14). Composes with `review_absorbing` (C5): Module 5 *narrows* what counts as a closing output, it never loosens the gate.
- **(f) New oracle: the implementation-verification attestation.** Someone attests that `implementationVerified` / `effectVerified` is true. The kernel checks the *lifecycle stage is recorded and cannot silently regress*, never that the effect was really achieved. **Contestation target M5-O6.**

Row 7 — Certificate validity (`Envelope` / `TEnvelope` reversibility certificates)

- **(a)** `Envelope.routeInside/sevInside : δ → Bool` (`TieredProtocol.lean:1361`) and `TEnvelope.routeOk/sevOk : σ → δ → Bool` with `Inv` (`1801`) — the certificates that a routing/severance action is "demonstrably reversible." v0.12 proves the gating *composes* (W21) but, per its own MANIFEST, "what the exposure state measures and whether the certified region describes the deployment's true reversible region" is **certification, at the seam** — outside the machine.

- **(b)** Track A (the certificate and its evidentiary basis); Track C (warnings that a certified-reversible action is in fact irreversible); Track F (custody of the certification process record); `03-adversarial-check` (the strongest case the certificate is *wrong*).
- **(c) The certified-lie failure — the deepest seam.** A certificate is a `→ Bool` an authority supplies. The v0.12 obligations (`route_preserves`) constrain the certificate to be *internally consistent* (it can't grant an exposure-exiting step), but nothing forces the certificate's `routeOk` predicate to track *physical* reversibility. An authority certifies an irreversible severance as reversible; the kernel's proofs then correctly conclude "reversible" from a false premise. The strongest theorems in the kernel rest here.
- **(d)** The wrapper attaches to the *certificate issuance*: a certified action carries a `SeamClaim` with the certifier's `authorizationChain`, the `evidenceRefs` grounding the reversibility claim, a mandatory `falsificationPath` (*what observation would show the action was in fact irreversible*), and `warningsAvailable` (dissent from the certification). Cost-to-fake of a false certificate rises from "return `true`" to "issue an attributed certificate that states its own falsifier and survives the attached adversarial check." The certificate can still be wrong — but a wrong certificate is now an *attributable, pre-falsified, contestable act*, not an anonymous Boolean.
- **(e)** *(Tier A)* `contested_cannot_unlock_irreversible`: a certificate `SeamClaim` whose `status = materiallyContested` (a surviving, evidence-linked challenge) cannot *independently* satisfy `authorizesC`/`TraceAuthorized` for an irreversible (Tier-3-class) action. **Composes directly with the v0.12 exposure layer (W19–W26):** the exposure invariant `Inv` gains a conjunct "no load-bearing certificate on this trace is materially contested." *(Tier A)* `certificate_is_evidence_backed` (analogue of Row 2 at the certificate seam).
- **(f) New oracle: the contestation-materiality relation.** Deciding a challenge is "material" (not frivolous) is a judgment. Per boundary §3 no classifier makes it; the kernel checks a challenge *exists and is evidence-linked* (`counterEvidence ≠ []`), and treats materiality as an *attested status*, never deciding it. **Contestation target M5-O7.** This is where the Forum-Now toggle's discipline enters: materiality is an *act* (a logged status flip), not a mood.

Row 8 — Criticality classification (`Event.critical`, Module 4)

- **(a)** `Event.critical : List Claim` with `critical_sub`/`falsifier_critical` (`PLOL.lean:197`) — the contestation-critical claims that must appear in every compliant register. v0.11's P4 (`residual_divergence_noncritical`) already flags this: §19.4.1's "framing, never findings" defense is "earned only under the additional assumption that the critical classification is COMPLETE … an extraction/Layer 0 obligation at the seam, stated in the module header, not a theorem."
- **(b)** Track A (the claim set and its extraction); Track D (whether criticality was used to *suppress* a claim); `01-evidence-matrix` (the propositions); the affected population's counter-register.
- **(c) The criticality-as-deletion-privilege failure.** The party that authors the `Event` decides which claims are `critical`. An authority marks a damaging claim *non-critical*, and every register may then omit it while remaining `Compliant`. The official criticality classification becomes a silent deletion privilege — the exact incompleteness P4 discloses but does not close.
- **(d)** The wrapper lets the **affected population attach claims to the same event hash** in a *counter-register* (`affectedCounterRecord`), independent of the official `critical` marking. Cost-to-fake of a suppressive classification rises from "omit from `critical`" to "omit while the affected population's counter-record, bound to the same digest, publicly asserts the omitted claim." The official record can still under-mark; it can no longer make the

claim *disappear* — the contradiction is preserved on the same hash (this is `bridge_proves_origin_not_consistency` P2 turned to the community's benefit).

- **(e)** *(Tier B)* `criticality_cannot_suppress_typed_conflict`: an `affectedCounterRecord` bound to an event's digest is not erasable by any official transition on that event, and its presence is decidable by the same mechanical check as P6 (`decCompliant`). The affected population's counter-register attaches to the event hash; official criticality is not a deletion privilege over it.
- **(f) New oracle: affected-population standing.** *Who* may attach a counter-record is a representation question. Per boundary §4, **weighted community standing enters only with governance machinery** (representation, revocation, intra-population disagreement); it is **not** stubbed with a Boolean. Until that machinery is modeled, the counter-register admits *any* attributed attachment and preserves it — standing to *weight* it is deferred, and the deferral is disclosed. **Contestation target M5-O8.**

Row 9 — Challenge dispositions (the response-must-be-on-the-merits input)

- **(a)** *(New in Module 5 — no v0.12 predecessor; this is the ledger's ADVERSARIAL CHECK / COUNTER-EVIDENCE STATUS discipline entering the kernel.)* A response to a contestation is currently *nothing* in the kernel — challenges have no typed home.
- **(b)** `03-adversarial-check` (mandatory field); the COUNTER-EVIDENCE STATUS field (§3 of the Standing Protocol); Track D (dismissal conduct).
- **(c) The interested-dismissal failure.** Without a typed disposition, a challenge can be waved away with no evidence and no record that it was answered at all — the "defeat of discernment itself" (Track D). This is where "well-typed lie" becomes "well-typed *dismissal*."
- **(d)** The wrapper types every challenge disposition as `accepted | rebuttedWithEvidence | unresolved | outsideScopeWithReason`, and a `rebuttedWithEvidence` disposition *must carry an* `EvidenceRef` (structure-field obligation). Cost-to-fake of a hollow dismissal rises from "ignore it" to "file a typed disposition that either concedes, cites rebutting evidence, admits it's unresolved, or states a scope reason — all on the record." **The machine verifies a linked disposition *exists*; it never verifies the disposition is *correct*.**
- **(e)** *(Tier B)* `challenge_requires_merits_response`: a challenge cannot be marked resolved without a disposition, and a `rebuttedWithEvidence` disposition is unconstructible with an empty evidence reference. Correctness of the rebuttal is explicitly *not* proved (labeled modeling disclosure).
- **(f) New oracle: the scope/merits judgment.** "Outside scope with reason" and "the rebuttal's evidence actually rebuts" are judgments. The kernel checks the *shape* (a reason string / an evidence ref is present), never the *substance*. **Contestation target M5-O9.**

Row 10 — The two-step standing boundary (Module 5's own non-negotiable)

- **(a)** *(New in Module 5 — the load-bearing safety property of the whole module.)* Community-triggered preservation/audit (Track D actions, step one) vs. authorized punishment (step two).
- **(b)** Standing Protocol §7 (the two-step position, and "the clause that does not narrow away"); §7-bis (Forum-Now: ACTIVE-soft / DORMANT-hard, "flipping is a logged act").
- **(c) The ledger-becomes-a-new-authority failure.** If a community-filed preservation demand could *itself* authorize a coercive measure, Module 5 would have imported a new unconstrained authority — the precise thing boundaries §1–

§2 forbid ("Veriticide is never an automatic trigger"). An allegation of discernment-corruption must not unlock coercion.

- **(d)** Not a cost-to-fake row — a *structural firewall*. Step-one objects (`preserve / disclose / audit / restrain-provisionally`) are a different type from step-two authorizations, and no function maps the former to the latter. There is nothing to fake because there is no path.
- **(e)** *(Tier A — treat as non-negotiable, per handoff §3)* `step_one_never_implies_ step_two`: no step-one `SeamClaim` (preservation/audit trigger) is accepted by any interface that authorizes a step-two (punitive/irreversible) action; the two are disjoint by type, proved by exhaustion over the disposition/action constructors. This is what prevents the ledger from becoming a new unconstrained authority.
- **(f) No new oracle.** This row *removes* a trust boundary rather than relocating one: it is a type-level disjointness, checkable with no external attestation. Its only modeling disclosure is the choice of which actions are "step two" (irreversible / punitive) — and that reuses v0.12's existing reversibility typing (`CAction`, `requiredTierC`), not a new oracle.

---

3. Summary: the ten new contestation targets (oracles introduced)

Per the reflexivity discipline, every relocation is named so a reviewer can attack it:

ID	Relocated trust boundary (new oracle)	Kernel checks	Kernel never checks	Boundary constraint
M5-O1	Authorization-chain attestation	chain exists, non-empty	members truly approved	—
M5-O2	Evidence extraction / grade	a graded ref exists	the grade is correct	—
M5-O3	Novelty baseline / prior-instance match	a falsifier is stated	the baseline is right	§3: no LLM classifier
M5-O4	Threshold config-to-deployment custody	a config digest is bound	it matches the running system	—
M5-O5	Actor-identity / conflict relation	a non-conflicted validator exists	who is "interested"	§3: no LLM classifier
M5-O6	Implementation/effect verification	lifecycle stage recorded, no silent regress	the effect was achieved	—
M5-O7	Contestation-materiality	a challenge is evidence-linked	the challenge is "material"	§3: no LLM classifier
M5-O8	Affected-population standing	an attributed attachment is preserved	who may *weight* it	§4: governance machinery required, else deferred
M5-O9	Scope / merits of a rebuttal	a disposition of the right shape exists	the rebuttal is correct	§3: no LLM classifier
—	*(Row 10 introduces none — it removes one)*	type disjointness	—	§1–§2: non-negotiable firewall

The map's own honest limit. Nine of ten rows *relocate* a trust boundary to a new oracle; the tenth *removes* one. Module 5 is therefore not a reduction in the number of trusted inputs — it is a **redistribution**: from single cheap anonymous bits to multi-party, dated, attributed, contestable objects whose falsification is a *documentary act on the record* rather than a private one. That redistribution is the entire claimed value, and it is honest exactly to the degree that column (f) is complete. If a reviewer finds an eleventh oracle Module 5 hides, that is a finding against this map.

---

4. Reflexivity disclosure (carried, per the framework's provenance rule)

The Veriticide corpus documents Anthropic among its subjects, and this map was prepared by an Anthropic model (`claude-opus-4-8`), operating IN-FRAMEWORK (Provenance Protocol Part I: framework documents in context; convergence value near zero as corroboration, documented-method value highest). The map's *claims* therefore carry no independent evidentiary weight. Its *value*, per Part II §5, must rest in externally checkable artifacts: the Lean invariants of column (e), which either compile under the pinned core-only toolchain with the published axiom footprint or do not; and the cost-to-fake analyses of column (d), which a reviewer can test against the ledger's own machinery. A GUID probe, a hash, and a Lean `#print axioms` line work the same regardless of who proposed them. This disclosure is made once, plainly, here.

---

5. What comes next (and what is gated on author direction)

Artifact 2 (the `SeamClaim` type + Module 5 skeleton) and artifact 3 (the triaged theorem program) follow the Tier A / Tier B split above. Two classes of decision are **author-gated** and are surfaced before the Lean is written, not decided unilaterally:

1. **Tier B relation modeling (M5-O5, M5-O7, M5-O8).** The actor-identity/conflict relation, contestation-materiality, and affected-population standing each require a *new modeled relation that is itself an oracle input*. Their shape (opaque attested predicate vs. structured record) is a modeling choice with a labeled disclosure. 2. **Deprecation timeline for the naked interfaces (handoff boundary §6).** Backward compatibility must not preserve a safety bypass. Whether v0.13 ships Module 5 *alongside* the naked `EpInput`/`Envelope` interfaces (with a deprecation schedule) or *gates* them behind the wrapper immediately changes the CI surface and the migration path.

A third, environment-level constraint is recorded here for the author: the pinned Lean toolchain (`leanprover/lean4:v4.15.0` via elan) **cannot currently be installed in this session** — the egress policy returns 403 on the GitHub release download. GitHub CI is unaffected (Actions installs its own elan), so Module 5 can be written and verified *by CI*, but not built locally in this environment as things stand. This raises the value of keeping each theorem small and the axiom-footprint expectations explicit, and is a reason to prefer landing Lean in reviewable increments.